XRPL Validator — Daily Engineering Report · 2026-08-24

Period: 2026-08-24 00:00 to 24:00 US Eastern (2026-08-24 04:00 to 2026-08-25 04:00 UTC). Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: xrpl-validator merge records (they begin 2026-07-04). Not available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records (they begin 2026-09-19); Step 1 receipt records (they begin 2026-08-29); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11); amendment data; data.xrpl.org agreement scores (scored days begin 2026-04-04; some days have none).

Status

Not assessed — no signing records for this day.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** no signing records for this day (they begin 2026-09-19).
- **Native engine:** no receipt records for this day (they begin 2026-08-29).
- **Changes:** 0 deployments, 0 commits merged into main.

Service levels

No signing records for this day (they begin 2026-09-19).

Native engine conformance

No receipt records for this day (Step 1 records begin 2026-08-29, Steps 2–5 records 2026-10-09).

Live counters are kept only for the current run; none exist for a past day.

Receipt register

No receipt records for this day (Step 1 records begin 2026-08-29).

Incidents

Not assessed: no signing records for this day.

Change log

Deployments

No deployment records before 2026-09-11.

Merged into main

No merges into main.

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **fix(xrpl-ledger): a partial fib slice prices at the slice's CEIL-16 encoded quality — one hundred twenty consecutive ledgers, the range is clear** (`9885998`, xrpl-validator) #106455151 52F632B8 (PHNIX→XRP→FUZZY bridge, multipath fib, five iterations): our partial takes of the pool's synthetic offers priced at the RAW s_in/s_out ratio (iters 3-4 came out 2 and 3 ulps light, 5e-9 of PHNIX total, the sender's line remainder the only visible key). rippled prices a partial like any offer — through the ENCODED quality — and the receipts prove it never re-curves (no SWAPOUT for the partial).
- **fix(xrpl-ledger): tearing down a redeemed LP line charges ONLY the holder — one hundred eighty-six consecutive ledgers** (`c88dc80`, xrpl-validator) #106455156 713DEF80 (full LP redemption, tfLPToken withdraw): mainnet threads the pool's AccountRoot untouched at OwnerCount 1 and takes the holder 80 → 79; tear_down_lp_line decremented BOTH sides and wrote the pool's root to 0. The pool is the LP token's ISSUER and never pays reserve on those lines — rippled's deleteAMMTrustLine adjusts the non-AMM side alone, the trustCreate charge-the-creator rule seen from the …
- **fix(xrpl-ledger): the delivered figure is rippled's actualOut — the sorted 16-digit fold, never an exact-width sum — one hundred ninety consecutive ledgers** (`8bef73f`, xrpl-validator) #106455221 34F37CD0 (no-partial XRP→CNY→XLM, two strands, four alternating iterations): every round matched rippled, the remainder chain hit zero — and the !partial judge still said tecPATH_PARTIAL, because the delivered pot accumulated at FULL width: the exact chain lands 80.905362410300298, 2e-15 short of the 80.9053624103003 Amount (round 2 an ulp low, round 3 the 16-digit remainder that cannot carry sub-ulp). …
- **fix(xrpl-ledger): crossing removes self-offers past the strict threshold — but only in an ADMITTED strand — one hundred ninety-four consecutive ledgers** (`5d46894`, xrpl-validator) Two coupled rippled rules, pinned by opposite-outcome specimens on the SAME book three ledgers apart: (a) The crossing stream steps PAST the strict threshold: `limitSelfCrossQuality` runs BEFORE `checkQualityThreshold` (BookStep.cpp:729/770),

both gating on qualityThreshold_ — the TRUE transfer-rate-inflated limitQuality — and "Remove this offer even if no crossing occurs" (:441-443). A failed flow keeps its …

- **fix(xrpl-ledger): the underfunded crossing clamp threads the balance as the walk's GROSS cap — two hundred six consecutive ledgers** (`5485685`, xrpl-validator) #106455229 7D1380A7 (IoC tfSell selling its whole 1.58026353300976e-5 BTC line through the XRP/BTC pool): the underfunded clamp computes tg_cross = avail/rate and hands the walk a NET budget with no gross cap, so the exhausting AMM slice re-grosses net x rate and overdrains the line by one ulp past zero (+1e-20 where mainnet writes canonical zero). rippled's clamped sendMax IS the crossing's remaining-in, and an …
- **fix(xrpl-ledger): trustCreate NoRipple — creator side from the tx flags ONLY, peer side stamped when the PEER lacks DefaultRipple — two hundred eleven consecutive ledgers** (`7ec3a7c`, xrpl-validator) #106455241 BD3CFFCF (a bot TrustSet with Flags=0 creating an XRPHAI line): our create arm defaulted the CREATOR's NoRipple from the CREATOR's own missing lsfDefaultRipple and never touched the peer side. rippled's trustCreate (RippleStateHelpers.cpp:258-283) does the opposite on both counts: the account-being-set gets NoRipple only from `bSetNoRipple && !bClearNoRipple`, and the PEER's side is stamped when the PEER …
- **fix(xrpl-ledger): the crossing walk's remainingOut is StrandFlow's fold, never a running subtraction — two hundred seventeen consecutive ledgers** (`cbfc29a`, xrpl-validator) #106455246 5BEBD5DB (7186 XRP buying 10514 RLUSD across forty quality levels): flowCross runs through flow() — one quality level (or one AMM slice) per iteration, the stream's level-change check breaking each pass (BookStep.cpp:720-724) — and every iteration re-derives remainingOut as outReq − the sorted-ascending 16-digit fold of savedOuts (StrandFlow.h:639-642). Our walk carried rem_pays at full width across all …
- **fix(xrpl-ledger): strand admission is judged once per iteration — an un-attempted self-removal resets the level anchor — TWO HUNDRED TWENTY CONSECUTIVE LEDGERS, THE WHOLE FETCHED RANGE** (`d94dc7f`, xrpl-validator) #106455252 3F75D634 (the counter-counter-specimen: two taker-owned offers on CONSECUTIVE levels, 1.86775e-11 and 1.86899e-11, both inside the inflated limit 1.870171055e-11): rippled removes BOTH inside flow iteration 0. The mechanism is the ofrQ reset — a self-removal with no offer yet attempted nulls the stream's level anchor ("if (!offerAttempted) ofrQ = std::nullopt", BookStep.cpp:441-448) — so the level-change …
- **fix(xrpl-ledger): the fwd chain carries the PRODUCED amount when the delta is exact, and a met-want hop flushes its excess input through the pool — two hundred thirty-nine consecutive ledgers** (`390bff3`, xrpl-validator) #106455266 CF7BAB85 (XRP -> OAR -> PLX, two pools, circular): pool 1's out rounds UP (29.71809727 OAR against the rev want ...26921203) and rippled's fwd hands the WHOLE product to book 2 — its pool receives the full figure, pays the curve's crumbs (70.7902709126 PLX), the dst is credited the Amount exactly and the ISSUER nets the 1.86e-9 difference. Two ports: (a) The hop carry prefers the measured delta over the …
- **fix(xrpl-ledger): the strand ub composes hop-0 trIn and the final DirectI's issuing rate — two hundred forty consecutive ledgers** (`af8bd2f`, xrpl-validator) #106455274 13296833 (24.29 USDT -> 5.92M PEPE, six strands): our strand upper bounds ran x1.001^2 too good, so strands 1 and 5 slipped past limitQuality 4.10227e-6 where rippled refuses them — turning rippled's single-admitted-strand, multiPath-false, one-anchored- iteration fill into three fib-sliced rounds with different intermediate fills. The two missing factors: (a) hop-0 trIn: the FIRST book step's prev is the …
- **fix(xrpl-ledger): AccountSet asf-to-lsf is a table, not a shift — two hundred fifty-eight consecutive ledgers** (`527956b`, xrpl-validator) #106455275 56049FD1 (SetFlag 13, asfDisallowIncomingCheck): mainnet stamps lsfDisallowIncomingCheck 0x08000000; our AccountSet wrote `1 << 13` = 0x2000. The asf numbers are enum

ordinals, not bit positions — the mapping is a table (LedgerFormats.h:128-142, TxFlags :408-425, ported verbatim): asf 1-4, 6-9 and 12-17 map to their lsf bits; asf 5 manages AccountTxnID FIELD PRESENCE (apply_common already rewrites it per …

Independence program

No independence record for UTC day 2026-08-24; records begin 2026-10-20.

Network

No ledger records for this day: the reference node holds ledgers from 2026-10-04 22:52 UTC (#107,436,647).

Amendment watch

No amendment records for this day.

Open items

None.

Corrections to earlier reports

None.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Concentrated Liquidity AMM Positions** (amm-defi) Add a new owner-reserved ltAMM_POSITION object (keylet::ammPosition(ammID, account, seq)) and optional sfTickLower/sfTickUpper SFields on AMMDeposit:
- **Credential-Gated Travel-Rule Payment Attachment** (compliance) Add a single optional sfTravelRuleData SField (opaque VL blob, capped at 2KB) attachable to Payment. Preclaim only accepts the field when the destinat
- **Bonded Oracle Slashing for Lending Feeds** (oracles) Add an sfOracleBond XRP-amount SField to the existing ltORACLE (PriceOracle) object, fundable via an optional field on OracleSet, held in ledger custo

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
08-24 04:23	—	xrpl-validator	`9885998`	fix(xrpl-ledger): a partial fib slice prices at the slice's CEIL-16 encoded quality — one hundred twenty consecutive ledgers, the range is clear
08-24 10:35	—	xrpl-validator	`c88dc80`	fix(xrpl-ledger): tearing down a redeemed LP line charges ONLY the holder — one hundred eighty-six consecutive ledgers
08-24 11:41	—	xrpl-validator	`8bef73f`	fix(xrpl-ledger): the delivered figure is rippled's actualOut — the sorted 16-digit fold, never an exact-width sum — one hundred ninety consecutive ledgers
08-24 14:09	—	xrpl-validator	`5d46894`	fix(xrpl-ledger): crossing removes self-offers past the strict threshold — but only in an ADMITTED strand — one hundred ninety-four consecutive ledgers
08-24 16:15	—	xrpl-validator	`5485685`	fix(xrpl-ledger): the underfunded crossing clamp threads the balance as the walk's GROSS cap — two hundred six consecutive ledgers
08-24 17:18	—	xrpl-validator	`7ec3a7c`	fix(xrpl-ledger): trustCreate NoRipple — creator side from the tx flags ONLY, peer side stamped when the PEER lacks DefaultRipple — two hundred eleven consecutive ledgers
08-24 18:21	—	xrpl-validator	`cbfc29a`	fix(xrpl-ledger): the crossing walk's remainingOut is StrandFlow's fold, never a running subtraction — two hundred seventeen consecutive ledgers
08-24 19:23	—	xrpl-validator	`d94dc7f`	fix(xrpl-ledger): strand admission is judged once per iteration — an un-attempted self-removal resets the level anchor — TWO HUNDRED TWENTY CONSECUTIVE LEDGERS, THE WHOLE FETCHED RANGE
08-24 23:15	—	xrpl-validator	`390bff3`	fix(xrpl-ledger): the fwd chain carries the PRODUCED amount when the delta is exact, and a met-want hop flushes its excess input through the pool — two hundred thirty-nine consecutive ledgers
08-25 01:21	—	xrpl-validator	`af8bd2f`	fix(xrpl-ledger): the strand ub composes hop-0 trIn and the final DirectI's issuing rate — two hundred forty consecutive ledgers
08-25 03:21	—	xrpl-validator	`527956b`	fix(xrpl-ledger): AccountSet asf-to-lsf is a table, not a shift — two hundred fifty-eight consecutive ledgers

Times in UTC. A commit appears on the day it was committed to main or merged into it.